DAY 1

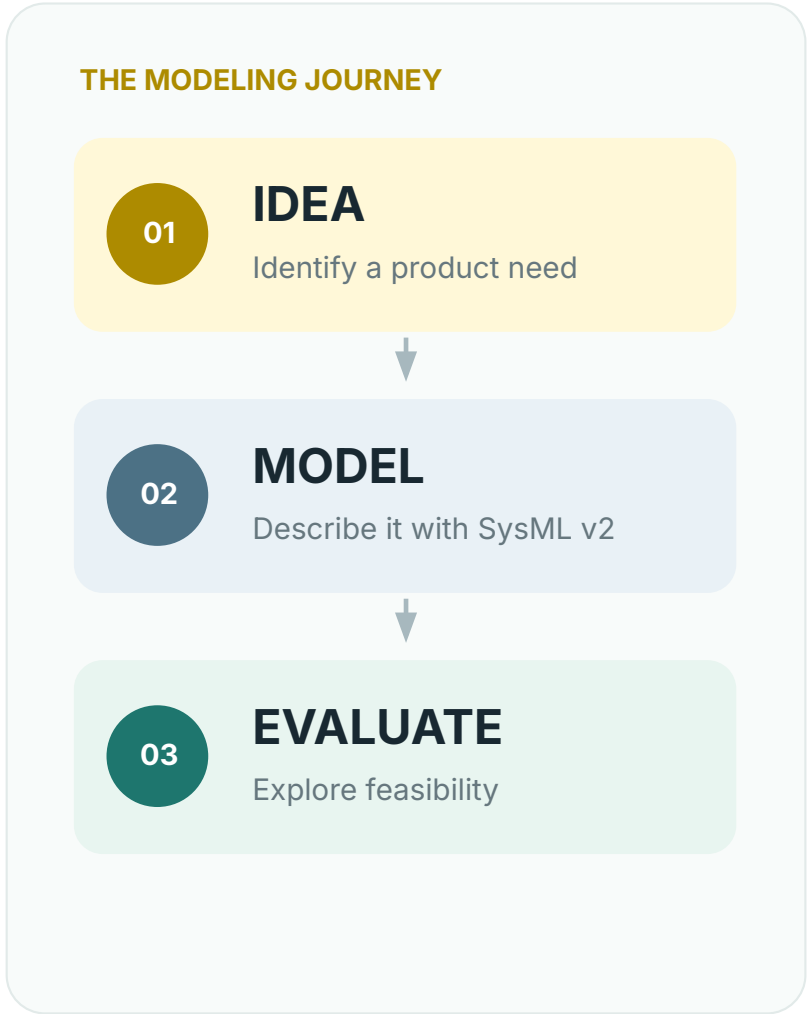
From Ideas to Products

The Evolution of Systems Modeling

SysML v2 / Model-based engineering / AI-assisted design

Laurent Balmelli · Hidekazu Nishimura

Keio University | October 2026



CLASS 1 / AGENDA

PERIOD 1

Class Schedule - Day 1

From ideas to products, then into our modeling environment.

1.1 Why modeling as a tool

90 MIN

- From ideas to products: the purpose of modeling
- Emergence of modeling languages and SysML
- What is SysML v2?

Break

15 MIN

1.2 Introduction to class tooling and installation

90 MIN

- The course toolchain: Git, Docker Engine, JupyterLab and OpenSysML
- Clone, build and launch the modeling workspace
- Create and display our first SysML v2 model

SECTION 1.1.1 / WHY MODELING?

4 MIN

Every product starts with an idea, not a design

An exciting concept quickly turns into a set of engineering questions.

01

IDEA

What if we could...

02

CONCEPT

What should it do?

03

MODEL

Could it work?

04

PRODUCT

Can we build it?

OUR RUNNING EXAMPLE

A smart bicycle light that senses darkness and saves power

CLASS QUESTION

What would you test first?

SECTION 1.1.1 / WHY MODELING?

4 MIN

Three tests determine whether an idea can succeed

Desirability, feasibility, and viability describe different kinds of uncertainty.

01

DESIRABILITY

Do people want it?

Would cyclists find an automatic light genuinely useful?

02

FEASIBILITY

Can we build it?

Could we make it compact and run for 10 hours?

03

VIABILITY

Does it make economic sense?

Could we manufacture and sell it at an acceptable price?

THIS COURSE ZOOMS IN ON FEASIBILITY

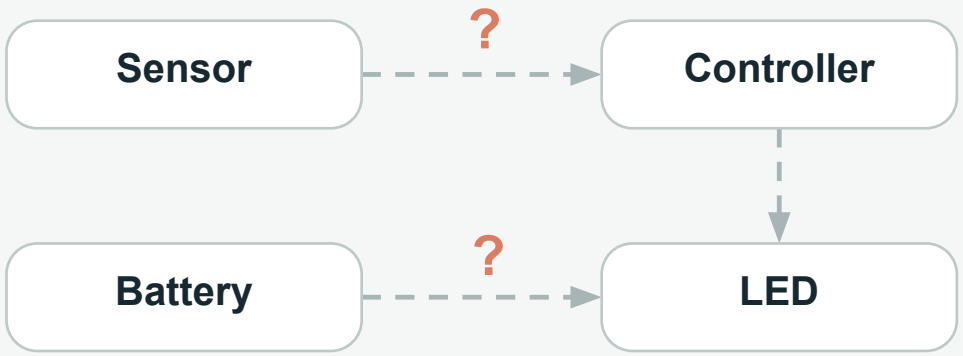
SECTION 1.1.1 / WHY MODELING?

5 MIN

A diagram shows shapes. A model captures meaning.

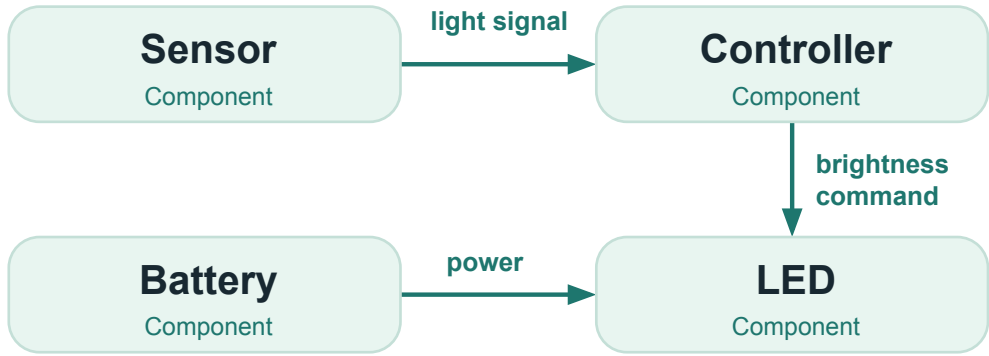
The same bicycle-light architecture can contain very different levels of information.

INFORMAL DRAWING



What do the arrows actually mean?

STRUCTURED MODEL



Components and connections have types and properties

Meaning enables checking, analysis, traceability, and reuse.

SECTION 1.1.1 / WHY MODELING?

5 MIN

Minimal Viable Models test feasibility early

Explore possible realizations, find gaps, and refine the concept before building.

PRODUCT CONCEPT

Smart bicycle light | target: 10 hours of operation

MVM A

Digital control

Sensor > microcontroller > LED

MVM B

Analog control

Sensor > analog circuit > LED

Two candidate implementations using different technologies

COMPARE FEASIBILITY GAPS — Battery life — Size — Control behavior

IF THE GAP IS TOO LARGE: revise the concept, implementation, or assumptions, then model again.

SECTION 1.1.1 / WHY MODELING?

2 MIN

In this course, you will move from ideas to evidence

Use models, tools, and AI assistance to reason about possible products.

01



IMAGINE

Define a product concept and its requirements.

02



MODEL

Describe structure, behavior, and connections in SysML v2.

03



INVESTIGATE

Explore alternatives, examine assumptions, and iterate.

NEXT: Why did engineers need standardized modeling languages?

SECTION 1.1.2 / HISTORY OF SYSML

2 MIN

From many modeling methods to a shared language

A common modeling language lets engineers express ideas consistently.

THE 1990s

Many competing methods

- Booch
- OMT
- OOSE

Different notations / inconsistent vocabulary

1997

UML

Unified Modeling Language

A shared vocabulary for software structure and behavior

Standardization improves communication and model exchange across teams and tools.

SECTION 1.1.2 / HISTORY OF SYSML

3 MIN

Why software-oriented UML was not enough

A physical product connects hardware, software, behavior, and requirements.

UML ORIGINS

Software modeling

Classes, objects, interactions, behavior

SYSML / THE ENTIRE BICYCLE LIGHT

ELECTRONICS

Battery, LED, sensor

SOFTWARE

Automatic switching

BEHAVIOR

Respond to darkness

REQUIREMENTS

10-hour battery target

ONE MODEL CONNECTS THE DISCIPLINES

SECTION 1.1.2 / HISTORY OF SYSML

4 MIN

SysML grew through two generations of standards

A community effort evolved from extending UML to redefining systems modeling.

2001 — **INCOSE + OMG**

Work starts on UML for systems engineering.

→

2003 — **Formal request**

OMG issues a systems engineering language RFP.

→

2005 — **Early SysML**

Specification drafts are submitted and reviewed.

2006 — **SysML v1**

Adopted in 2006; formally published in 2007.

→

2017 — **SysML v2 RFP**

The next-generation language effort begins.

→

2025 — **SysML v2**

OMG approves and publishes the new standard.

FROM A UML PROFILE TO A NEW FOUNDATION

SECTION 1.1.2 / HISTORY OF SYSML

3 MIN

SysML v1 described four connected perspectives

Different model views describe the same engineered system.

REQUIREMENTS

What must it achieve?

Battery life >= 10 hours

STRUCTURE

What is it made of?

Battery / sensor / controller / LED

BEHAVIOR

How does it operate?

Switch on when ambient light is low

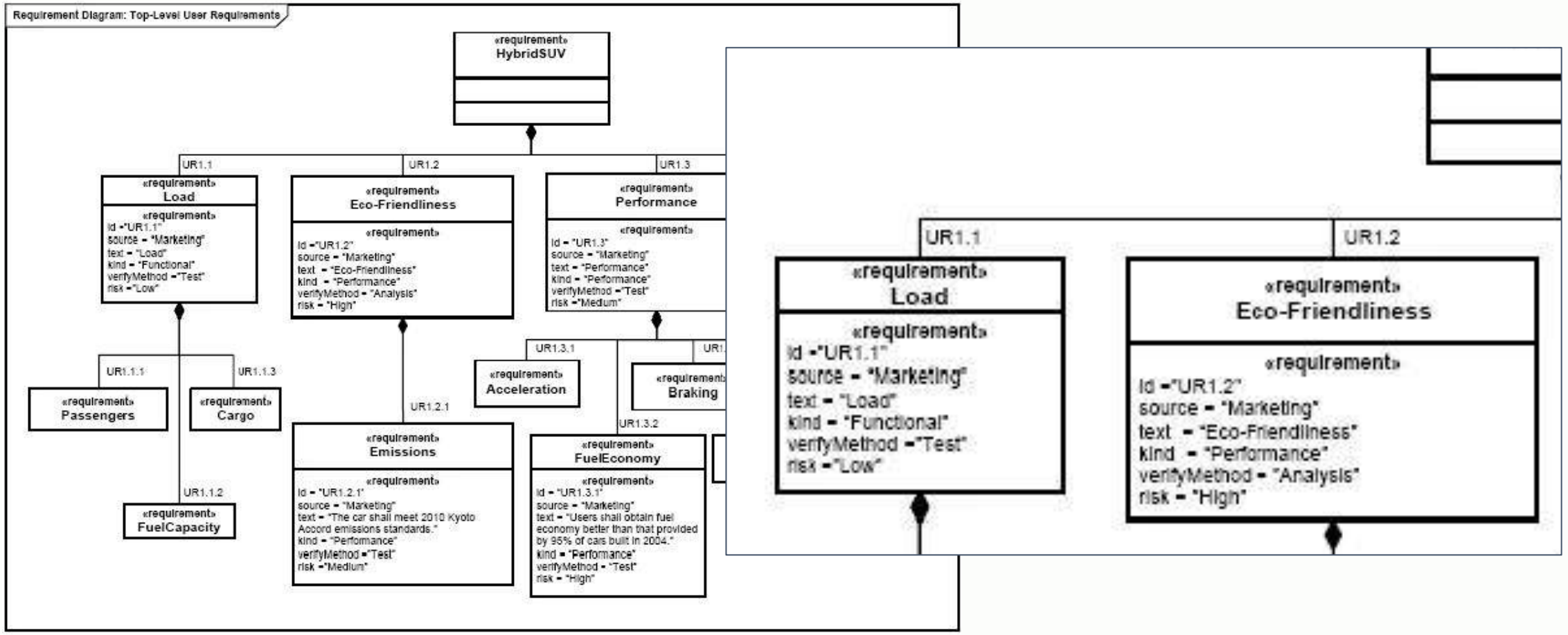
PARAMETRICS

Which quantitative rules apply?

Energy use / power / runtime

→ **NEXT: SysML v2 makes models more precise, reusable, and accessible to automation.**

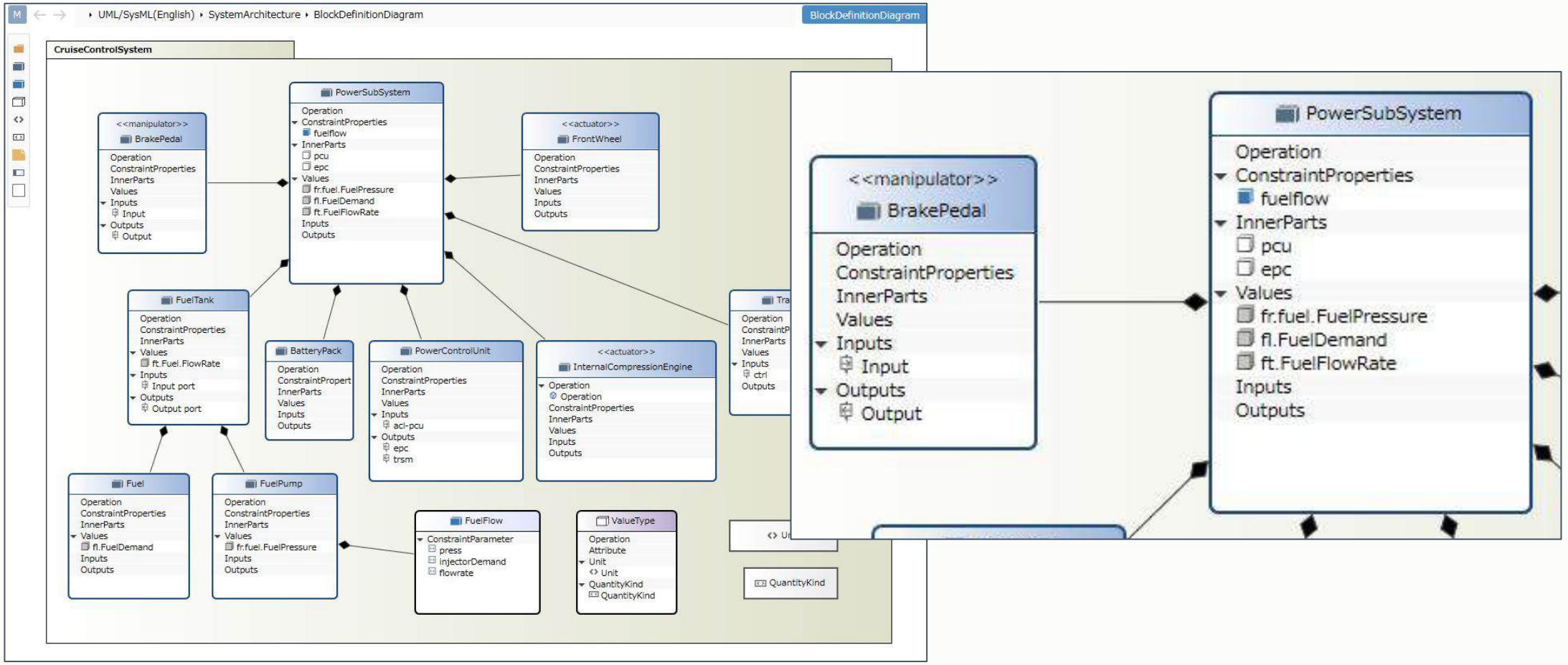
SysML v1 REQUIREMENTS - What must it achieve?



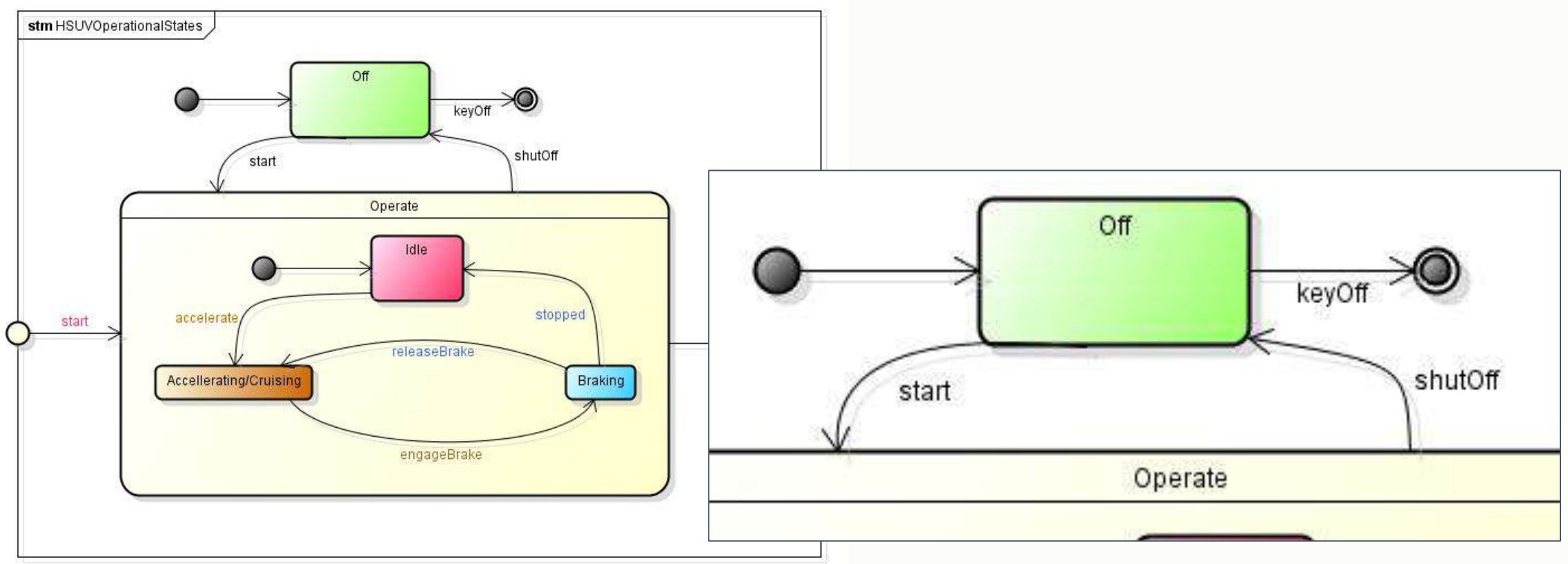
SECTION 1.1.2 / HISTORY OF SYSML

3 MIN

SysML v1 STRUCTURE - What is it made of?



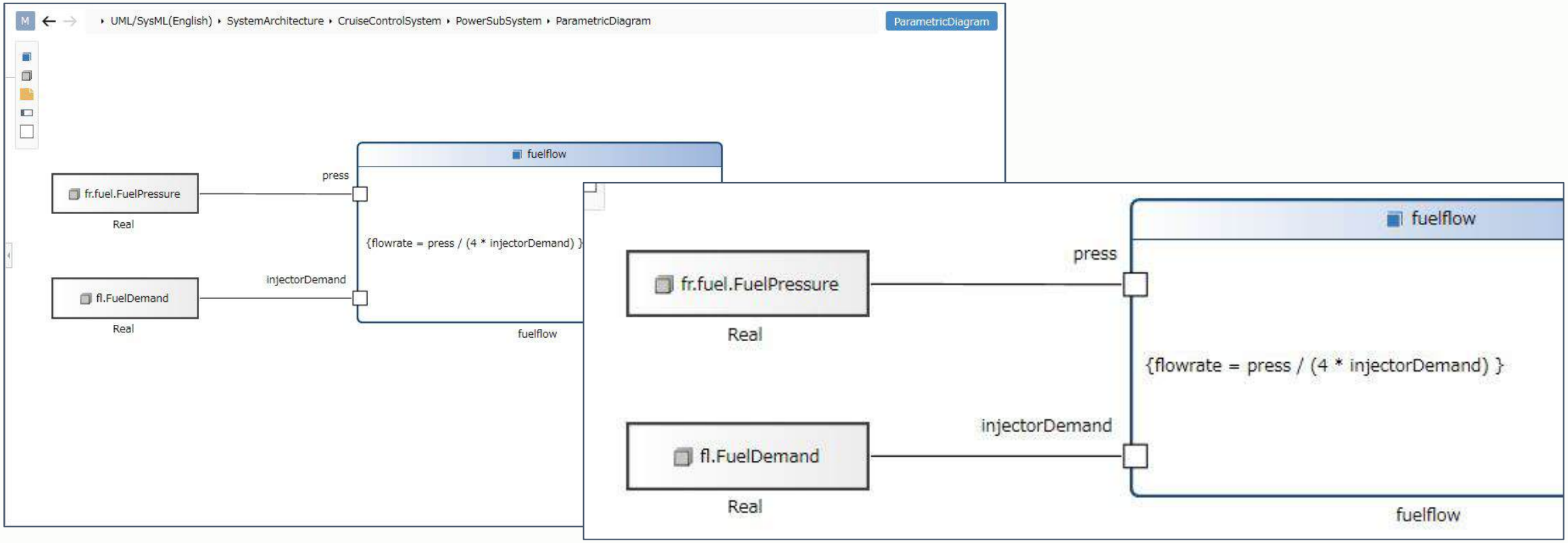
SysML v1 BEHAVIOR - How does it operate?



SECTION 1.1.2 / HISTORY OF SYSML

3 MIN

SysML v1 PARAMETRICS - Which quantitative rules apply?



SECTION 1.1.3 / WHAT IS SYSML v2?

2 MIN

SysML v2 is a new foundation, not just new diagrams

The second generation replaces the UML-profile foundation with KerML.

SYSML v1

SYSML v1 PROFILE

↓

UML 2

An extension of software-oriented UML

SYSML v2

SYSML v2 LANGUAGE

↓

KerML

A dedicated kernel with precise semantics

WHY IT MATTERS — **More precise meaning enables consistent interpretation and automation.**

SECTION 1.1.3 / WHAT IS SYSML v2?

3 MIN

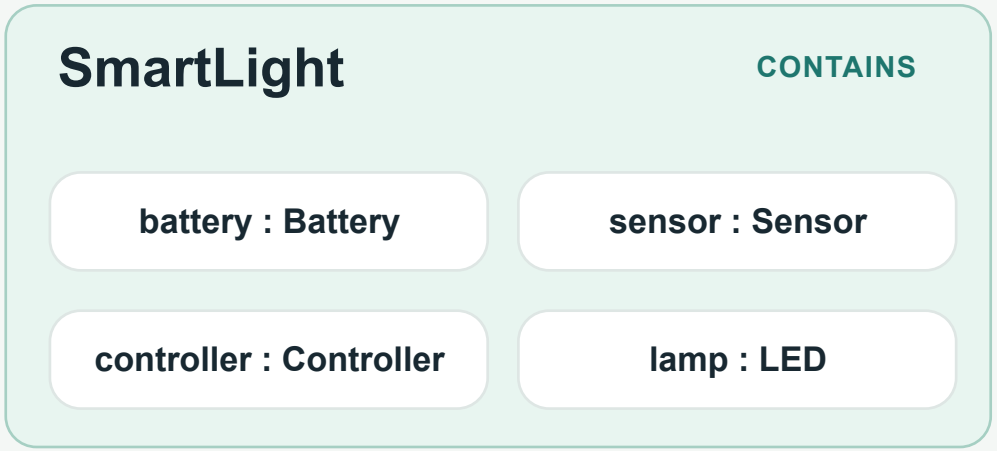
One semantic model, two ways to express it

SysML v2 defines both textual and graphical representations.

</> TEXTUAL MODEL

```
part def Battery;
part def Sensor;
part def Controller;
part def LED;
part def SmartLight {
  part battery : Battery;
  part sensor : Sensor;
  part controller : Controller;
  part lamp : LED;
}
```

CONCEPTUAL GRAPHICAL VIEW



TEXT: easy to edit and compare versions **GRAPHICS: easy to explain and explore**

SECTION 1.1.3 / BICYCLE LIGHT / 1 OF 6

4 MIN

01. Build the bicycle-light structure

Definitions and part usages describe what the product contains.

PASTE INTO OPENCODE

Create a SysML v2 model of an automatic bicycle light. Define reusable Battery, LightSensor, Controller, and LED parts. Define SmartBicycleLight containing one of each, and one bicycleLight usage.

Create a notebook called day1-demo and a model BicycleLight-day1.sysml and add a renderable BicycleLightViews::overview view. Check the syntax against local OpenSysML examples.

LOOK FOR • Definitions, contained parts, and one system instance

SECTION 1.1.3 / BICYCLE LIGHT / 2 OF 6

4 MIN

02. Add the engineering properties

Attributes give our components measurable characteristics.

PASTE INTO OPENCODE

Extend the model without changing existing parts. Add properties for usable battery energy (30 Wh), LED average power (3 W), measured ambient light, and the darkness threshold.

Use SysML v2 attributes and supported quantity/unit syntax. If the installed OpenSysML kernel lacks a feature, choose a simpler valid representation and explain the limitation. Keep BicycleLightViews::overview usable.

LOOK FOR • Numerical properties attached to the right components

SECTION 1.1.3 / BICYCLE LIGHT / 3 OF 6

4 MIN

03. Connect the components

Ports, connections, and flows describe how components exchange resources and information.

PASTE INTO OPENCODE

Extend the model with interfaces: the battery supplies electrical power to the controller and LED; the sensor sends ambient-light measurements to the controller; the controller sends an on/off command to the LED.

Use supported SysML v2 ports, connections, and flows. Keep energy exchanges distinct from information exchanges. Preserve the model and renderable view; explain any unsupported notation.

LOOK FOR • Power paths are different from information paths

SECTION 1.1.3 / BICYCLE LIGHT / 4 OF 6

4 MIN

04. Describe automatic light control

Behavior captures how the product changes in response to its environment.

PASTE INTO OPENCODE

Add automatic-control behavior to the model. In bright conditions the LED is Off. When ambient light falls below the threshold it turns On; when light rises above the threshold it returns Off.

Use SysML v2 actions, states, and transitions supported by the installed OpenSysML version. Preserve the earlier declarations and explain how we can inspect the behavior in Jupyter.

LOOK FOR • Events and conditions that move between Off and On

SECTION 1.1.3 / BICYCLE LIGHT / 5 OF 6

4 MIN

05. Say what the product must achieve

Requirements express obligations independently of the chosen implementation.

PASTE INTO OPENCODE

Add SysML v2 requirements to the same model: the light must switch on automatically when it is dark, and it must operate for at least 10 hours on a fully charged battery.

Define the requirements explicitly and relate them to relevant system elements where supported. Preserve the complete model and view. Explain which requirements are still unverified.

LOOK FOR • Requirements distinct from components and behavior

SECTION 1.1.3 / BICYCLE LIGHT / 6 OF 6

4 MIN

06. Test a feasibility assumption

A calculation turns properties and requirements into an engineering question.

PASTE INTO OPENCODE

Extend the model with a simple runtime estimate: usable battery energy 30 Wh divided by average power 3 W gives 10 hours. Represent the calculation and compare it with the 10-hour requirement using supported SysML v2 constructs.

Update a view to expose the bicycle-light system and relevant analysis elements. Keep the file loadable in Jupyter, and distinguish a computed estimate from physical validation.

LOOK FOR • A 10-hour estimate, a requirement, and an explicit assumption

SECTION 1.1.3 / WHAT IS SYSML v2?

2 MIN

Define a component once, use it in many places

SysML v2 separates reusable definitions from their contextual usages.

DEFINITION

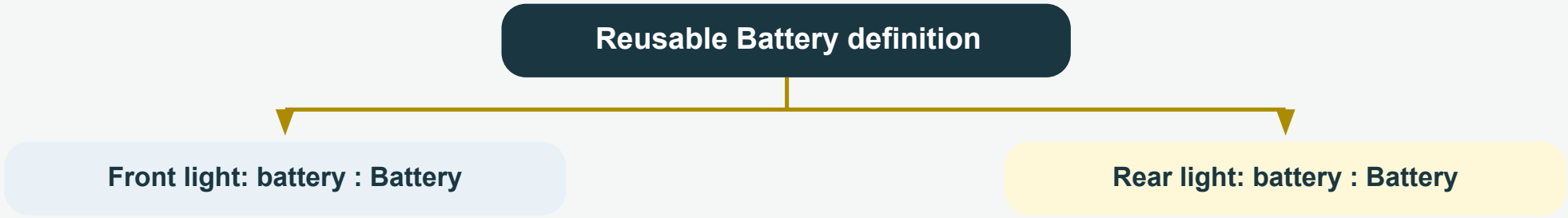
part def Battery;

A reusable component type

USAGE

part battery : Battery;

A role within a particular product



A common definition improves consistency and reduces duplication.

SECTION 1.1.3 / WHAT IS SYSML v2?

3 MIN

A model can help us test feasibility assumptions

Properties, expressions and constraints support quantitative reasoning.

RUNTIME ESTIMATE

Runtime = usable battery energy / average power

(hours = Wh / W) Assumes constant average power and fully usable energy.

DESIGN A

24 Wh / 3 W

8 hours

BELOW THE 10-HOUR TARGET

DESIGN B

30 Wh / 3 W

10 hours

MEETS THE SIMPLE ESTIMATE

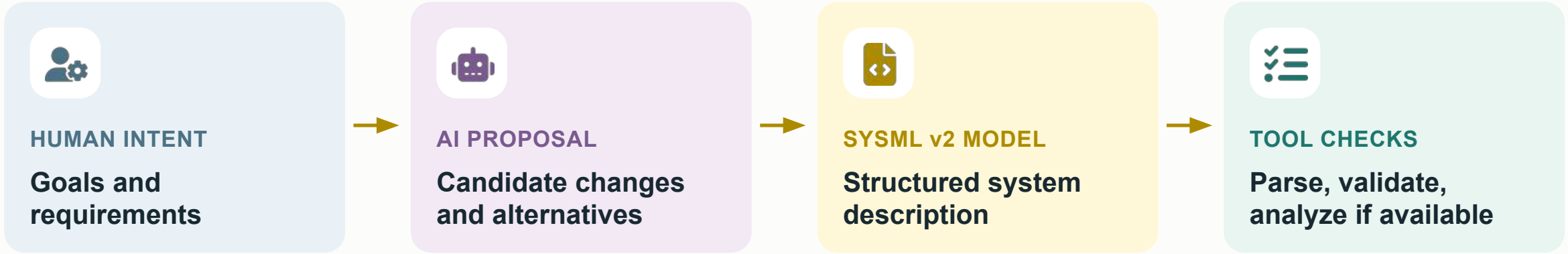
DESIGN QUESTION: could we lower power use instead of increasing battery size?

SECTION 1.1.3 / WHAT IS SYSML v2?

3 MIN

AI can propose a model. Engineers must judge it.

A productive workflow combines machine-readable models with checks and review.



REVIEW AND ITERATE

The engineer accepts, rejects or revises proposals.

STANDARDIZED INTEGRATION

APIs and OSLC can connect lifecycle tools.

→ **NEXT: install the course environment, then begin with first_model.md**

SECTION 1.2 / TOOLING & INSTALLATION

3 MIN

One workspace for every modeling exercise

Git retrieves the course; Docker Engine runs the tools; JupyterLab lets us explore models.

01

COURSE REPOSITORY

Git + course files

Documentation, examples, notebooks

02

DOCKER ENGINE

Reproducible runtime

Builds and runs the pinned image

03

JUPYTER + OPENSYSML

Interactive modeling

Execute SysML and render views

TODAY'S GOAL

Write, run and render one SysML v2 model.

SECTION 1.2 / TOOLING & INSTALLATION

4 MIN

Prepare Git, Docker Engine and Compose v2

This course uses open-source Docker Engine, not Docker Desktop.

WINDOWS

WSL2 + Ubuntu

```
wsl --install
```

Install Docker Engine and Compose inside Ubuntu.

macOS

Colima Linux VM

```
colima start --runtime docker
```

Install Colima, Docker CLI and plugins with Homebrew.

LINUX

Native Docker Engine

```
sudo systemctl enable --now docker
```

Install Engine, CLI, Buildx and Compose v2.

CHECK `git --version` `docker --version` `docker compose version`

SECTION 1.2 / TOOLING & INSTALLATION

8 MIN

Clone the course and build the toolchain

The Makefile provides the supported commands; work in the repository root.

HOST TERMINAL

```
git clone https://github.com/lbalmelli/keio-2026.git keio_2026

cd keio_2026

make samples

make build
```

PROJECT FOLDER

```
keio_2026/
  README.md
  compose.yaml
  Dockerfile
  Makefile
  docs/first_model.md
  notebooks/
  workspace/samples/
```

make samples prepares the OpenSysML checkout | make build creates the pinned Jupyter image

SECTION 1.2 / TOOLING & INSTALLATION

5 MIN

Start JupyterLab and select the SysML kernel

One command starts the environment; use your browser to open the notebook.

HOST TERMINAL

make up

make up remains attached to the terminal.

In another terminal:

make logs | make down

JUPYTERLAB IN THE BROWSER

http://localhost:8888/lab

DEFAULT TOKEN **opensysml**

Create a notebook Select SysML v2 (OpenSysML)

Notebook files live in the persistent notebooks/ folder.

SECTION 1.2 / TOOLING & INSTALLATION

5 MIN

Follow the README.MD file

Agentic Product Concept Modeling 2026
Laurent Balmelli, Hidekazu Nishimura
vers 11, 12/10/26

Course workspace and getting started

Welcome to **Agentic Product Concept Modeling 2026**. This repository contains the course material and the local tooling used for hands-on product concept modeling.

You will explore how product ideas can be described with **SysML v2**, a systems-modeling language, and practiced through **Jupyter** notebooks. The course also considers AI-assisted design: AI can help us explore, explain, and refine concepts while the system model keeps the product structure explicit. The workspace provides Docker, Jupyter, OpenSysML, and an optional OpenCode assistant inside JupyterLab.

Current tooling

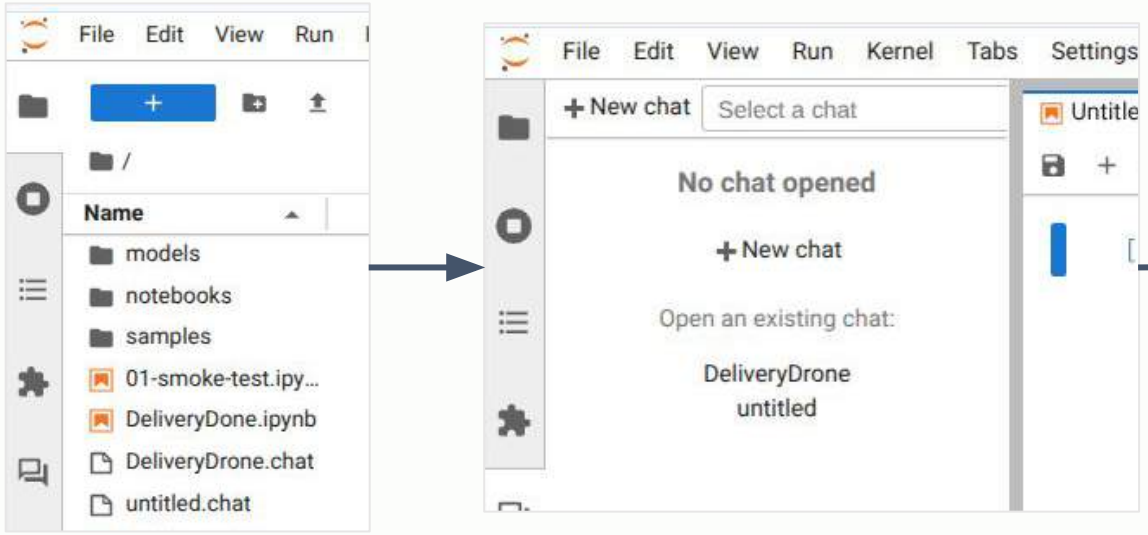
Tool	Purpose	Runs
Docker Engine	Provides the Linux container runtime.	Locally on Linux, inside WSL2 on Windows, or inside a Linux VM on macOS.
Docker Compose	Builds and starts the course services from `compose.yaml`.	Locally as a host terminal command.
JupyterLab	Provides the browser-based notebook environment.	Inside the Docker container.
OpenSysML Jupyter kernel	Executes SysML v2 cells and renders declared views.	Inside the Jupyter container.
Jupyter AI 3.2.0	Provides the native chat panel and Jupyter notebook MCP tools.	Inside the Jupyter container.
OpenCode 1.19.0	Provides a SysML-focused ACP persona; students authenticate with their chosen provider.	As a CLI subprocess of Jupyter, under the `keio` user.

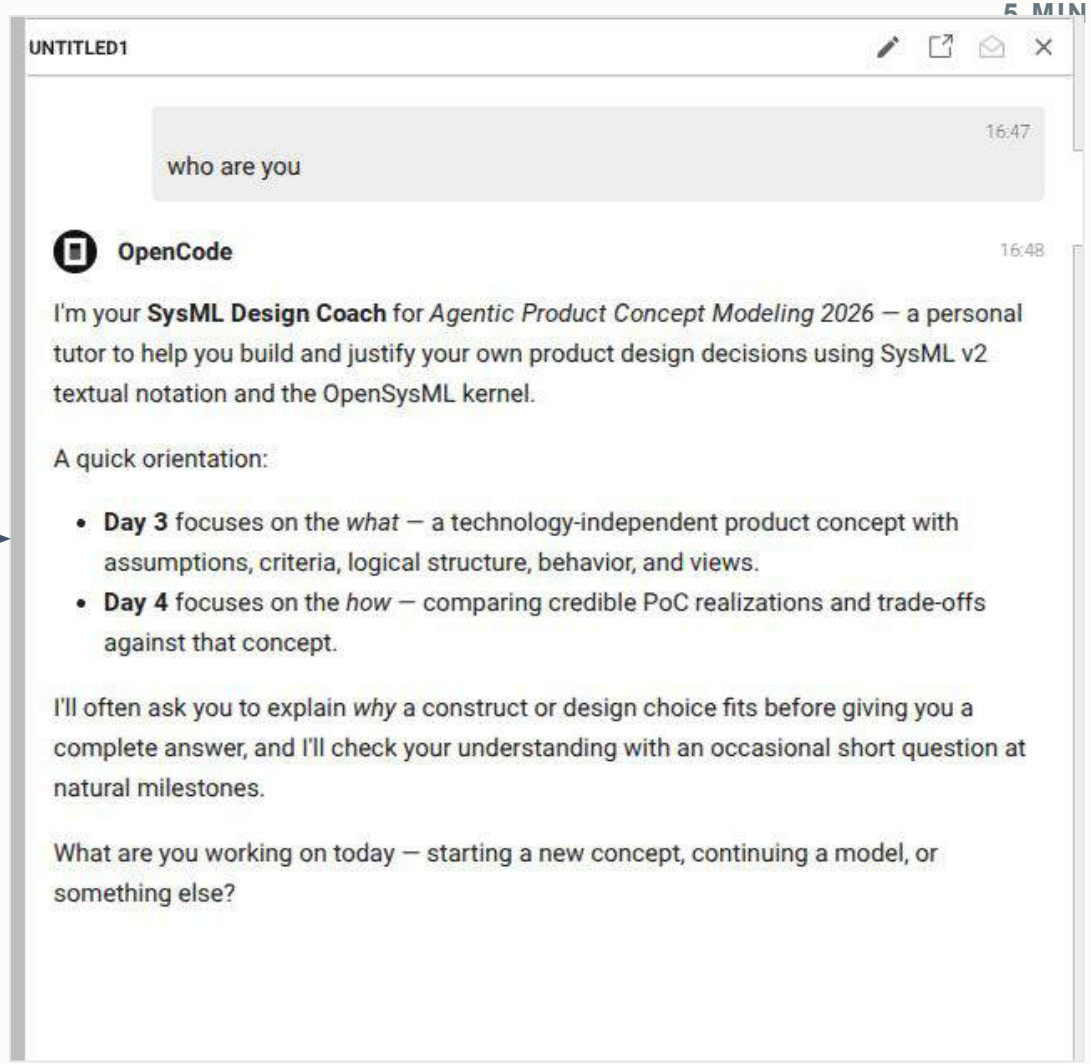
The OpenSysML kernel is pinned to `0.9.3.dev20261008`. Jupyter AI is pinned to `3.2.0`. OpenCode is pinned to `1.19.0`; its Linux amd64 and arm64 release archives are checked against SHA-256 digests during the image build. There is one browser interface at `http://localhost:8888/lab`; OpenCode does not start a web server or expose another port.

SECTION 1.2 / TOOLING & INSTALLATION

5 MIN

Set-up your environment





SECTION 1.2 / TOOLING & INSTALLATION

5 MIN

Meet your new coach

16:51

can you create a notebook for me

OpenCode 16:51

Happy to! Let me check the workspace layout so I put it in the right place.

✓ ls work 2>/dev/null; ls /workspace 2>/dev/null – Always allow

I can see there's a `notebooks/` folder — I'll create a fresh notebook there for you.

✓ Jupyter_MCP_Server_create_notebook – Always allow

I've created **`notebooks/concept-work.ipynb`** and opened it in JupyterLab. It currently has one empty code cell, ready for OpenSysML kernel work (`%load` , `%print` , `%render`).

A few things to know:

- New models get saved as distinct `.sysml` files under `/workspace/models` — we won't touch the read-only samples.
- If you want strict syntax checking later, we can run `%strict on` and submit declarations in a kernel cell.

What would you like to work on first — starting a Day 3 concept from a problem scenario, or do you already have a model in progress?

16:53

This is still Day 1, I am trying to understand how this works. What can I do in Jupyter?

OpenCode 16:53

Welcome to Day 1! Let me give you a tour of what this Jupyter environment can do for you in this course.

What you can do in Jupyter here

1. Run SysML models live The notebook uses an OpenSysML kernel. You write SysML v2 declarations in a code cell and execute it (Shift+Enter). Typical commands:

- `%load <filename>` — load a SysML file into the kernel
- `%print` — print the current model state
- `%render` — render readable views of the model
- `%strict on` — strict syntax checking (great for learning; it surfaces parser diagnostics)

2. Write and iterate on models You type SysML v2 textual notation directly into cells, e.g. defining parts, attributes, requirements, state machines, or activities — and see parse results immediately instead of waiting for a report.

3. Explore sample models There's a read-only `/workspace/samples` folder with example models you can study and load — we never edit those, but they're a great reference.

SECTION 1.2 / TOOLING & INSTALLATION

7 MIN

First model: introduction to your first model

Follow docs/first_model.md; run model, view and render cells in order.

</> 1 DEFINE THE MODEL

```
package KeioVehicle {
    part def Wheel;

    part def Vehicle {
        part wheels[4] : Wheel;
    }

    part car : Vehicle;
}
```

2 DECLARE A VIEW

```
view completeVehicleModel {
  expose KeioVehicle; ...
```

3 RENDER THE VIEW

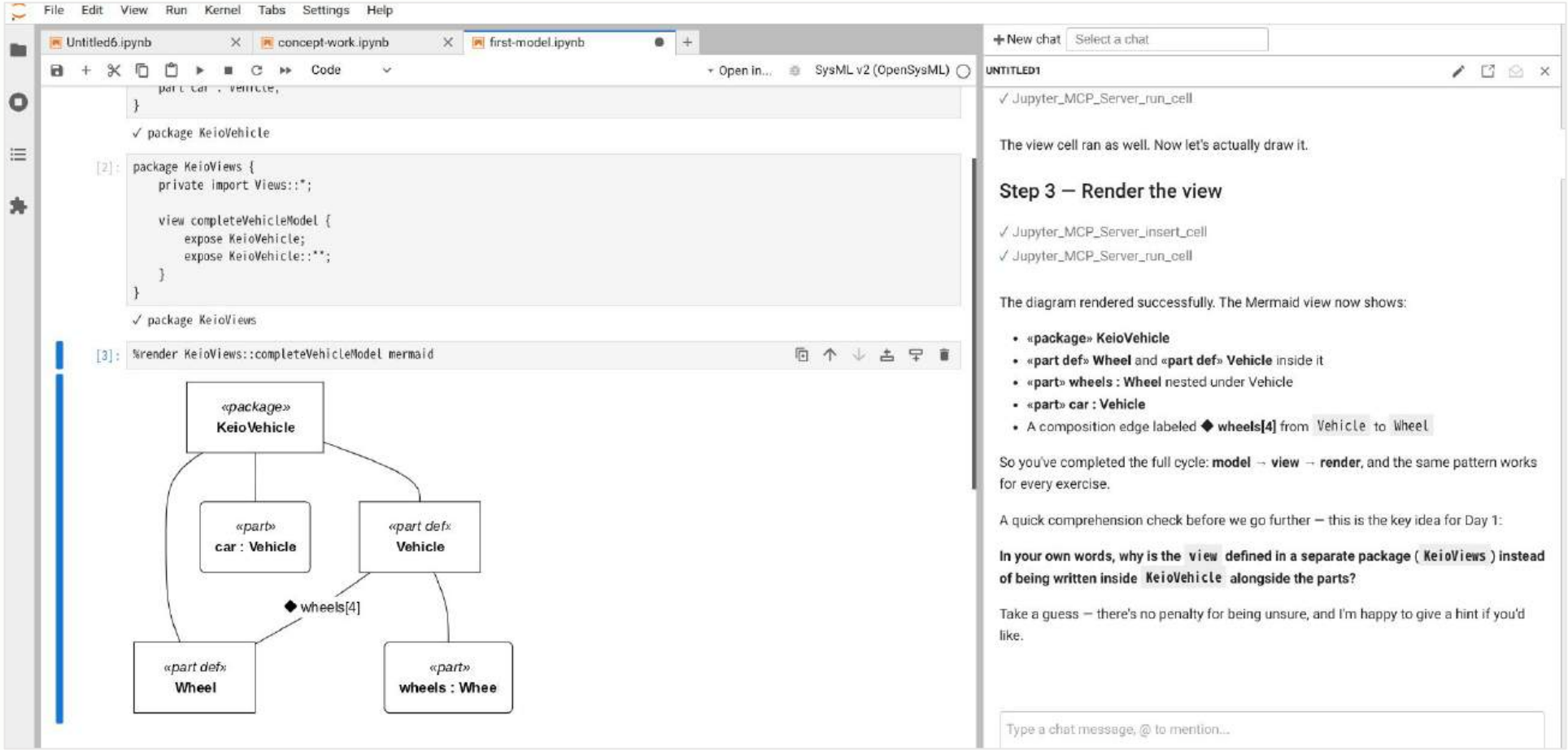
```
%render KeioViews::completeVehicleModel mermaid
```

SUCCESS: OpenSysML renders a diagram of the model in your notebook.

SECTION 1.2 / TOOLING & INSTALLATION

7 MIN

First model: do it all with your coach



SECTION 1.2 / TOOLING & INSTALLATION

7 MIN

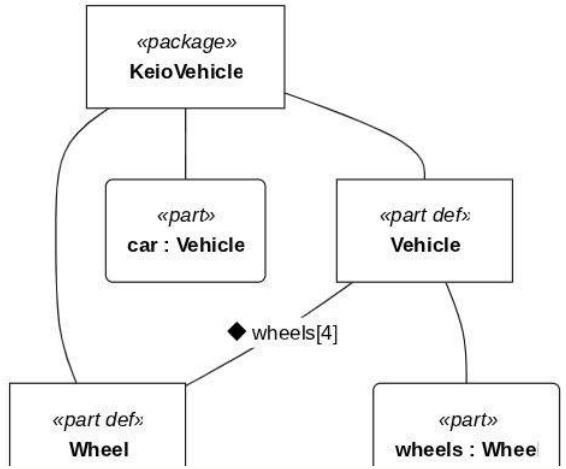
First model: the coach will ask you questions

```
[2]: package KeioViews {
         private import Views::*;

         view completeVehicleModel {
             expose KeioVehicle;
             expose KeioVehicle::**;
         }
     }
```

✓ package KeioViews

```
[3]: %render KeioViews::completeVehicleModel mermaid
```



for every exercise.

A quick comprehension check before we go further — this is the key idea for Day 1:

In your own words, why is the `view` defined in a separate package (`KeioViews`) instead of being written inside `KeioVehicle` alongside the parts?

Take a guess — there's no penalty for being unsure, and I'm happy to give a hint if you'd like.

17:09

Because it allows to create a many-to-one relationship between views and model. In other words we can choose series of you to render a model or part of it.

OpenCode 17:10

Exactly right — that's the core idea. Let me sharpen it slightly:

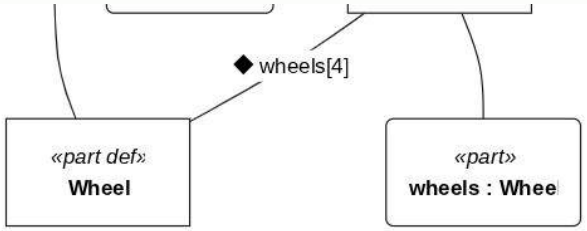
- **Many views, one model**: each view is a *selection* over the same underlying model, so you can render the whole `KeioVehicle` package or just a slice of it, and produce as many views as you need.
- **Direction note**: from the model's perspective it's one model → many views (1:N); from each view's perspective it's many views → one model (N:1). Your phrasing captures it well either way.
- **Separation of concerns**: the model stays about the *product*, while `KeioViews` stays

Type a chat message, @ to mention...

SECTION 1.2 / TOOLING & INSTALLATION

7 MIN

First model: Use it to perfect your understanding - views

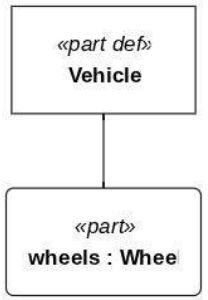


[4]:
```
package KeioViews2 {
    private import Views::*;

    view vehicleOnlyModel {
        expose KeioVehicle::Vehicle;
        expose KeioVehicle::Vehicle::**;
    }
}
```

✓ package KeioViews2

[5]:
```
%render KeioViews2::vehicleOnlyModel mermaid
```



Yes let's add a second View and let's compare the two views

OpenCode

Let's do it. I'll add a second view package that exposes **only** `Vehicle` (plus what it contains), leaving out the top-level `car` and the standalone `Wheel` declaration.

✓ Jupyter_MCP_Server_insert_cell
✓ Jupyter_MCP_Server_run_cell
✓ Jupyter_MCP_Server_insert_cell
✓ Jupyter_MCP_Server_run_cell

Both rendered. Here's the side-by-side comparison — **the same model, seen through different lenses:**

	`completeVehicleModel`	`vehicleOnlyModel`
Package `KeioVehicle`	✅ shown	❌ not shown
`part def Wheel`	✅ shown	❌ not shown
`part def Vehicle`	✅ shown	✅ shown
`part wheels : Wheel`	✅ shown	✅ shown
`part car : Vehicle`	✅ shown	❌ not shown (not exposed
Composition edge ◆ wheels[4]	✅ shown	✅ (implied by nesting)

Two things worth noticing:

慶應義塾 Keio University

SECTION 1.2 / TOOLING & INSTALLATION

7 MIN

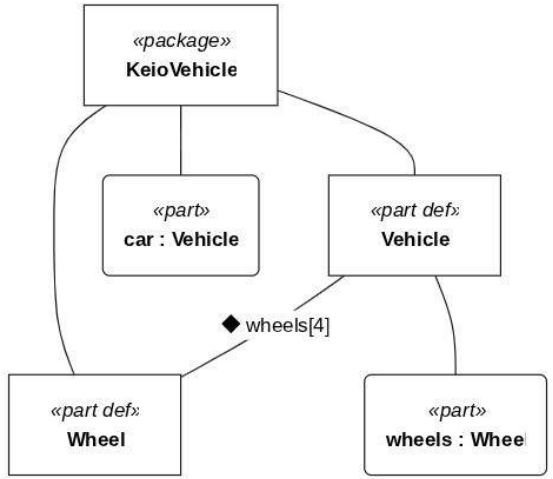
First model: Use it to perfect your understanding - notation

Want me to save this walkthrough model as a `.sysml` file under `/workspace/models` so you have a record? Or would you prefer to stop here and pick it up later?

17:14

yes save it. Can you explain the notation between the parts Wheel and Vehicle? In particular the diamond notation.

[3]:
```
%render KeioViews::completeVehicleModel mermaid
```



[4]:
```
package KeioViews2 {
    private import Views::*;
```

Now, the diamond — great question, this is one of the most important notations in SysML.

The diamond is **composite aggregation** (composition)

In the render, the edge between `Vehicle` and `Wheel` was shown as:

```
◆ wheels[4]
```

and the diamond sits at the **`Vehicle` end**. That tells you three things at once:

1. Ownership / whole–part relationship `Vehicle` is the *whole*; `wheels` is the *part*. The diamond is always drawn at the **whole** end, never the part end. If you ever see the diamond next to the "wrong" element, the model means something different.

2. Lifetime coupling (why it's "composite") In SysML v2, a `part` usage declared inside a part definition is **composite by default**. That means `Vehicle` *owns* its `wheels` : the parts live and die with the whole. Conceptually, if the `Vehicle` is destroyed, its `wheels` go with it — they aren't shared elsewhere.

3. Multiplicity `[4]` says exactly four wheels. Combined with the diamond, it reads: *"a Vehicle is composed of exactly four Wheel parts."*

The contrast you'll meet next: shared aggregation

If you run into problems: check the README.MD (Git repo)

Common problems

Symptom	Likely cause	Solution
`docker: command not found`	Docker is not installed or the CLI is not on `PATH`.	Install Docker Engine and the Compose plugin using the platform instructions above, then open a new terminal and repeat `docker --version`.
Docker says the daemon is not running	Docker Engine or its Linux VM runtime is stopped.	Start Docker Engine, WSL2, or Colima as appropriate for your platform, then verify it with `docker info`.
`docker compose` is unavailable	The Compose v2 plugin is missing or the Docker installation is incomplete.	Install or enable Docker Compose v2, then verify with `docker compose version`.
Permission denied when using Docker on Linux	Your user cannot access the Docker daemon socket.	Use the Docker permission setup for your Linux distribution, start a new login session, and verify with `docker info`. Do not change the repository files to work around host permissions.
Port `8888` is already in use	Another service is using the default Jupyter host port.	Run `export JUPYTER_PORT=8889`, then `make up`, and open `http://localhost:8889/lab`.
OpenCode is missing from the persona picker	Image is stale, CLI is missing, or the Jupyter AI extension failed to load.	Run `make build`, restart with `make down` then `make up`, and run `make doctor`.
OpenCode persona has no response	Provider credentials or model are unavailable.	Run `make opencode-auth`, then select an authenticated provider/model in Chat.
Agent cannot read the notebook or sees old content	No active notebook/browser session or the wrong notebook tab is active.	Select the intended notebook and cell, then ask the agent to identify them. Live MCP behavior requires an open JupyterLab browser.

SECTION 1.2 / TOOLING & INSTALLATION

3 MIN

Ready for the next class: model as code

Check the environment, preserve your notebook, and explore a richer model later.

STUDENT SUCCESS CHECKLIST

- Course repository cloned locally
- make samples and make build completed
- JupyterLab opens at localhost:8888/lab
- OpenSysML kernel selected
- SysML model displayed as a diagram

NEXT EXERCISE

official_vehicle_model.md

Load and view an official SysML v2 example.

IF ANYTHING FAILS

make logs **make down**

Help: command + full error + OS details.

SAVE YOUR WORK IN notebooks/ BEFORE STOPPING THE ENVIRONMENT

DAY 1 / END

Model the idea. Question the model.

A model makes an idea testable.

Describe / Analyze / Iterate / Validate

Questions?

NEXT / DAY 2: MODEL-AS-CODE

THREE KEY TAKEAWAYS

01 **WHY MODEL?**
Explore feasibility early

02 **WHY SYSML v2?**
A shared system language

03 **WHY TEXT + AI?**
Generate, check, and refine